

Orgams Binary Format Specification

Krusty/Benediction & copilot

Abstract

This document specifies the Orgams binary format, used for storing z80 assembler source code in a compact binary representation. The format preserves source code structure, expressions, and formatting information while enabling efficient storage and reconstruction of assembler programs, particularly for Amstrad CPC demo development.

Introduction

The Orgams binary format is designed to store z80 assembler source code in a compact binary representation that maintains fidelity to the original source formatting and structure. This format is primarily used by the Orgams assembler tool in the context of Amstrad CPC demoscene development.

This specification defines the binary encoding of assembler instructions, expressions, directives, labels, and control structures. The format uses a chunk-based structure with specialized encoding for different types of source code elements.

Requirements Language

The key words “MUST”, “MUST NOT”, “REQUIRED”, “SHALL”, “SHALL NOT”, “SHOULD”, “SHOULD NOT”, “RECOMMENDED”, “MAY”, and “OPTIONAL” in this document are to be interpreted as described in RFC 2119 {{RFC2119}}.

Terminology

- **Item:** A discrete element of source code (instruction, directive, label, etc.)
- **Marker:** A byte value that identifies the type of item
- **Expression:** A mathematical or logical expression
- **Chunk:** A section of the file containing related data
- **SizedString:** A length-prefixed string
- **Bit7OnString:** A string terminated by a byte with bit 7 set

File Structure

An Orgams binary file consists of the following components in sequence:

1. Header
2. Source Code Chunk (SRCc)
3. Null Separator (0x00)
4. Labels Chunk (LBLs)
5. Null Separator (0x00)
6. Checksum Chunk (ChCk)

All multi-byte values are stored in little-endian byte order.

Header

The header consists of:

- Magic bytes: 4 bytes (0x4F 0x52 0x47 0x41) “ORGA”
- Version: 1 byte (currently 0x02)
- Header size: 1 byte indicating additional header data length
- Header data: Variable length data of specified size

Chunks

Each chunk begins with a 4-byte ASCII identifier:

- “SRCc” (0x63 0x52 0x43 0x53): Source code data
- “LBLs” (0x73 0x4C 0x42 0x4C): Label definitions
- “ChCk” (0x6B 0x43 0x68 0x43): Checksum data

Chunks are separated by null bytes (0x00).

Source Code Chunk (SRCc)

The SRCc chunk contains the main source code as a sequence of items. Each item represents a construct in the assembler source.

Item Encoding

Items are encoded using marker bytes that indicate their type. Some markers are used directly, while others require an escape prefix.

Direct Markers

The following marker bytes are used directly without prefix:

Marker	Value	Description
MARKER_NEWLINE	0x4A	Line terminator
MARKER_COMMENT	0x43	Comment - followed by length-prefixed string
MARKER_INDENT	0x49	Indentation - followed by space count
MARKER_ASSIGN	0x64	Assignment - followed by label and expression
MARKER_BYTE	0x42	BYTE directive - followed by expression list
MARKER_WORD	0x57	WORD directive - followed by expression list
MARKER_MACRO_DEF	0x46	Macro definition - followed by macro data
MARKER_LABEL_REF	0x60-0xDF	Label reference - followed by label encoding

Escaped Commands

Commands are prefixed with the escape marker 0x7F:

Command	Value	Description
CMD_ASIS	0x01	Raw string - followed by length-prefixed string
CMD_IF	0x09	IF statement - followed by expression
CMD_ELSE	0x0A	ELSE statement
CMD_END	0x0B	END statement
CMD_BRK	0x0C	BRK statement
CMD_RESTORE	0x0D	RESTORE statement
CMD_FILL	0x0E	FILL directive - followed by size and value expressions
CMD_ENT	0x0F	ENT directive - followed by expression
CMD_ORG	0x10	ORG directive - followed by expression
CMD_ORG2	0x11	ORG2 directive - followed by two expressions
CMD_SKIP	0x12	SKIP directive - followed by expression
CMD_IMPORT	0x13	IMPORT directive - followed by length-prefixed string
CMD_ENDM	0x14	ENDM statement
CMD_STORE_PC_INSTR	0x15	Store PC instruction marker

Command	Value	Description
CMD_STORE_PC_LINE	0x16	Store PC line marker
CMD_REPEAT	0x17	REPEAT statement - followed by expression and item
CMD_MACRO_USE	0x18	Macro call - followed by macro data
CMD_END_BIS	0x19	END_BIS statement

Instruction Encoding

Z80 instructions are encoded using their native opcodes. Instructions with opcodes that conflict with markers are prefixed with 0x7F (escape byte).

Instruction format:

[escape|prefix] opcode [operand0] [operand1] ...

Where: - **escape**: Optional 0x7F byte if opcode conflicts with a marker - **prefix**: Optional instruction prefix byte (see table below) - **opcode**: 1-byte z80 instruction opcode - **operands**: Zero or more expressions (each as length + expression bytes)

Prefix Bytes

Extended instructions use prefix bytes:

Prefix	Value	Purpose
IX_CODE	0xDD	IX register operations
IY_CODE	0xFD	IY register operations
MARKER_IX_IND	0xDF	IX indexed addressing
MARKER_IY_IND	0xFF	IY indexed addressing
EXTENDED_CODE	0xED	Extended instructions
BIT_CODE	0xCB	Bit operations

For indexed addressing (0xDF/0xFF), the format is: prefix + displacement_byte + opcode

The displacement is a signed byte (-128 to +127) that immediately follows the prefix.

Instruction Examples

- PUSH AF: 0xF5 (no prefix, no operands)
- PUSH IX: 0xDD 0xF5 (IX prefix + opcode)
- LD A, (IX+5): 0xDF 0x05 0x7E (IX indexed prefix + displacement + opcode)

- LD A,42: 0x3E 0x01 0x30 0x2A (opcode + expression: 1 byte length + decimal 8-bit marker + value)
- Escaped instruction (if opcode is 0x4A): 0x7F 0x4A (escape + opcode)

String Types

Length-Prefixed String (Len + String)

Format: **length_byte** + **length** bytes of content

- First byte: length (0-255)
- Following bytes: string content
- Content uses Windows-1252 encoding
- Used for comments, strings, import paths

Example: “Hello” → 0x05 0x48 0x65 0x6C 0x6C 0x6F

Bit-7 Terminated String

Format: sequence of bytes where the last byte has bit 7 set to 1

- All bytes except last: normal character values (0x00-0x7F)
- Last byte: character value OR 0x80 (bit 7 set)
- When reading: clear bit 7 from last byte to get actual character
- Used for labels and identifiers in string table
- Terminator detection: `byte & 0x80 != 0`

Example: “AB” → 0x41 0xC2 (A=0x41, B=0x42|0x80=0xC2)

Expressions

Expressions represent mathematical and logical operations. They consist of sequences of ExpressionMembers.

Expression Types

- **Sized Expression:** 1-byte length + expression bytes (total length = length byte value)
- **Unsize Expression:** variable length expression data with implicit termination

Expression Members

Short Decimal Constants

- 0x00-0x1F: Direct encoding of values 0-31

Operators

Operator	Value	Symbol
EXP_OP_PLUS	0x2B	+
EXP_OP_MINUS	0x2D	-
EXP_OP_TIMES	0x2A	*
EXP_OP_DIV	0x2F	/
EXP_OP_MOD	0x25	%
EXP_OP_AND	0x26	&
EXP_OP_OR	0x7C	
EXP_OP_XOR	0x5E	^
EXP_OP_EQ	0x3D	=
EXP_OP_LT	0x3C	<
EXP_OP_GT	0x3E	>
EXP_OP_PAREN_OPEN	0x28	(
EXP_OP_PAREN_CLOSE	0x29	)
EXP_SPACE	0x20	space

Numeric Values

Type	Marker	Format
Decimal 8-bit	0x30	marker + 1 byte
Decimal 16-bit	0x31	marker + 2 bytes (little-endian)
Decimal custom	0x32-0x33	marker + length + length bytes
Hexadecimal 8-bit	0x34	marker + 1 byte
Hexadecimal 16-bit	0x35	marker + 2 bytes
Hexadecimal custom	0x36-0x37	marker + length + length bytes
Binary 8-bit	0x38	marker + 1 byte
Binary 16-bit	0x39	marker + 2 bytes
Binary custom	0x3A-0x3B	marker + length + length bytes

Labels and Special

Element	Value	Description
Label reference	0x60-0xFF	Label index
EXP_DOLLAR	0x24	\$ symbol
EXP_DOUBLE_DOLLAR	0x44	\$\$ symbol
EXP_STRING	0x53	String in expression - followed by length-prefixed string

Special Elements

Element	Value	Description
EXP_ITER1	0x41	Iteration marker 1
EXP_ITER2	0x42	Iteration marker 3
EXP_ITER3	0x43	Iteration marker 3

Multi-term Expressions

Element	Value	Description
EXP_MULTI_TERM_BEGIN	0x42	Begin complex expression
EXP_MULTI_TERM_END	0x45	End complex expression

Multi-term expressions are enclosed between begin (0x42) and end (0x45) markers.

Expression Examples

- 42: 0x30 0x2A (decimal 8-bit: 42)
- 0x100: 0x35 0x00 0x01 (hexadecimal 16-bit: 0x0100)
- label + 1: label_index 0x2B 0x01 (label + short decimal 1)
- (a + b) * 2: 0x42 label_a 0x2B label_b 0x45 0x2A 0x02 (multi-term)

Labels Chunk (LBLs)

The LBLs chunk contains label definitions:

- Version byte: 0x02
- Sequence of bit-7 terminated strings
- Each string followed by null separator (0x00)
- Labels are referenced by index (0-based) in expressions

Example: Two labels “A” and “MY_LABEL”:

0x02 0xC1 0x00 0x4D 0x59 0x5F 0x4C 0x41 0x42 0xC5 0x4C 0x00

- 0x02: version
- 0xC1: “A” with bit 7 set
- 0x00: separator
- 0x4D...0xC5: “MY_LABE” + “L” with bit 7 set
- 0x00: separator

Checksum Chunk (ChCk)

Contains validation data:

- 4 bytes: CRC or checksum of file content

Display Formatting

Source reconstruction uses state-based formatting:

Line States

- Empty: Start of line
- AfterLabel: Following a label
- AfterStatement: Following a statement
- AfterRepeatBloc: In repeat block
- AfterIf: After IF statement
- AfterOrg: After ORG directive

Formatting Rules

State	Prefix
Empty	TAB_COMMAND/TAB_INSTR spaces
AfterLabel	TAB_COMMAND/TAB_INSTR spaces
AfterStatement	“.”
AfterRepeatBloc	TAB_COMMAND spaces
AfterIf	“.”
AfterOrg	“.”

TAB_INSTR = 10 spaces, TAB_COMMAND = 6 spaces.

Examples

Simple Instruction

PUSH AF

Encoded: 0xF5 0x4A

Comment

; Hello World

Encoded: 0x43 0x0C 0x48 0x65 0x6C 0x6C 0x6F 0x20 0x57 0x6F 0x72
0x6C 0x64

IF Statement

```
IF condition
  instruction
ELSE
  other
END
```

Encoded: 0x7F 0x09 expression_bytes 0x7F 0x0A 0x7F 0x0B

Security Considerations

This format does not contain executable code and poses no direct security risks. However, as with any binary format, implementations should validate input data to prevent buffer overflows or malformed data processing.

IANA Considerations

This document has no IANA actions.

Implementation Notes

- Parsing uses Winnow combinator library
- Display reconstruction aims for exact source fidelity
- Windows-1252 encoding for text content
- Little-endian byte order throughout

— back

Acknowledgments

This specification is based on the implementation in `binary_decoder.rs` of the `cpclib-orgams-ascii` crate.

References

Author's Address

Romain Duroselle Email: romain@cpcdemotools.org